GREAT SALT LAKE DUST & AIR QUALITY MODEL

Understanding the Model: Physics, Math, Geology, and How to Judge the Results

A plain-language walkthrough built to bring you up to speed on everything the project does — the lake science behind it, how the equations were built term by term, and how to tell whether a result is trustworthy — so you can start extending it yourself with real understanding, not just running scripts.

explanation_model.pdf · companion to docs/MODEL_EXPLAINER.md and the project's living doc · prepared September 2026

Contents

1. Why this project exists
2. Great Salt Lake & dust, 101 — the geology and physics behind every equation
3. From an idea to an equation — building the model term by term
4. Fitting the model — what "calibration" actually means
5. How to read the results, and how to tell if they're legit
6. What the model actually found, in plain English
7. Where to go next

1. Why this project exists

Great Salt Lake is shrinking. It has no outlet to the ocean — water flows in from rivers and snowmelt, and leaves only by evaporation — so as more water gets diverted upstream for farms and cities, and as drought reduces how much arrives in the first place, the lake drops and its historic bed is left exposed. In November 2022 it hit the lowest level ever recorded. A big snow year in 2023 brought it back up somewhat, but it remains far below healthy historical levels.

That exposed lakebed — called **playa** — is not solid ground. It's fine lake sediment: silt, clay, and salts left behind as the water retreated. When wind blows hard enough across dry playa, it picks up that sediment and launches fine dust particles into the air. Those particles are small enough to travel many miles and be inhaled deep into human lungs. This is a real, active public-health and policy question for the Salt Lake Valley, not an abstract modeling exercise.

> **THE CORE QUESTION THIS PROJECT ANSWERS**
>
> How much of the PM10 (a specific size class of airborne particle) measured at a monitor in Salt Lake City can be attributed to dust blowing off the exposed Great Salt Lake bed, as opposed to traffic, construction, wildfire smoke, or dust from other dry regions — and how does that change with wind, atmospheric conditions, and how low the lake is?

This document assumes you know none of the geology, the math, or the statistics going in. Each idea is built up from scratch, in order, so that by the end you can read the model's code and results and know what you're looking at.

2. Great Salt Lake & dust, 101

2.1 What kind of lake is this?

Great Salt Lake is a **terminal lake** in an **endorheic (closed) basin**: water flows in, but the only way out is evaporation. Nothing ever flushes it downstream. Over thousands of years, this means dissolved minerals just keep concentrating — which is why the lake is many times saltier than the ocean. It's the largest remnant of a much larger Ice Age lake, Lake Bonneville, that once covered much of western Utah.

2.2 Why does a shrinking lake create a dust problem?

Think of the lakebed as a giant, shallow bowl. As the water level drops, more of the bowl's rim is exposed as dry land. That newly-exposed sediment was, until recently, sitting under water — it's loose, fine-grained, and often high in salts. Given enough wind, it erodes just like any other dry, unvegetated soil. The lower the lake, the more exposed playa there is available to erode.

2.3 The physics of how wind actually lifts dust (saltation)

This is the single most important physical idea behind the model's math, so it's worth getting right. Wind blowing over dry ground does *not* smoothly lift fine dust straight into the air. Instead:

1. Once wind speed near the surface crosses a **threshold**, it starts rolling and bouncing larger, sand-sized grains along the ground. This hopping motion is called **saltation**.
2. Each time a bouncing grain lands, it acts like a tiny hammer, knocking much finer dust particles loose from the surface. Those fine particles are light enough to get lifted into the air and carried downwind.
3. Below the threshold wind speed, none of this happens — the surface just sits there, however dusty it looks.

> **WHY THIS MATTERS FOR THE MATH**
>
> This is exactly why the model's emission term is not simply "more wind = more dust." It's a **threshold** process: nothing below a cutoff wind speed, then a rapidly increasing amount above it. That's what the term `max(wind speed − threshold, 0) raised to a power` in Section 3 represents — it's a direct translation of saltation physics into an equation.

2.4 Why moisture (and salt crust) shuts emission off

Wet sediment doesn't erode — the water holds grains together. Even after the surface dries, salts left behind can cement the top layer into a hard crust that resists wind erosion much better than loose, dry silt. This is why recent rainfall measurably suppresses dust emission for a period afterward, and it's why the model includes a term that decays emission based on how much it has rained in roughly the last two days.

2.5 Why PM10, specifically?

"PM10" means particulate matter smaller than 10 micrometers across — about a seventh the width of a human hair. That size cutoff isn't arbitrary: particles this small can be inhaled past the body's natural filtering (nose hair, mucus) and reach deep into the lungs, which is why regulators track it as a health indicator rather than, say, total dust by weight. A closely related measure, PM2.5 (smaller still), is dominated by combustion sources like wildfire smoke and vehicle exhaust, while coarser dust tends to show up more in the PM10-and-larger range. That distinction is exactly what lets a future version of this model start telling "this spike was dust" apart from "this spike was wildfire smoke" — see Section 7.

2.6 Why wind direction and distance matter, not just wind speed

Dust has to physically travel from the lakebed to the monitor to be measured there. If the wind is blowing from the east while the lake sits to the northwest of the monitor, any dust kicked up over the lake gets carried away from the city, not toward it — the monitor would see nothing, no matter how much dust the lake is emitting. And even when the wind direction is right, the trip takes time: dust emitted over the lake doesn't appear at the monitor instantly, it arrives after however long it takes the wind to carry it there. Both of these ideas — direction and travel time — are explicit terms in the model, built in Section 3.

3. From an idea to an equation

3.1 The "bathtub" intuition

Before looking at any equation, hold this picture in your head: imagine the air above the monitor as a bathtub of a fixed size. The water level in the tub is the PM10 concentration. Things can add water (fill the tub) or drain it (empty the tub), and the level right now depends on the level a moment ago, plus whatever got added or removed since then. That's the entire idea behind a **state equation** — a formula for "what happens in the next moment, given what's true right now."

In our case, the "moment" is one hour, and the things filling or draining the tub are:

- **Background air flowing in** (fills the tub toward whatever level the surrounding region already has)
- **Lake dust arriving** (fills the tub, only when wind, direction, and dryness line up)
- **Local sources** like traffic and construction (fills the tub steadily)
- **Removal** — dust settling out, getting rained out, etc. (drains the tub)

3.2 The actual equation

```
P[t+1] = P[t] + dt × { k_v(t)·[P_bg(t) − P[t]] + E_GSL(t−tau)/H(t) + E_local −
lambda·P[t] }
```

Read left to right: **next hour's PM10** equals **this hour's PM10**, plus **everything that changed in between**. `dt` is just "one hour," included so the units work out. Now each piece of the "everything that changed" bracket, one at a time:

3.3 Term 1 — background air

```
k_v(t) · [ P_bg(t) − P[t] ]
```

P_bg(t) is the "background" PM10 level — roughly, what the air would look like on an unremarkable day with no lake event, no rush hour spike, nothing unusual. The model estimates it directly from the data: the lowest 10% of PM10 readings over the trailing 30 days.

k_v(t) is a ventilation rate — think of it as how fast a window is open. If `P[t]` is above background, this term pulls it back down toward background; if below, it pulls it up. A bigger `k_v` means faster mixing with the surrounding air (a wide-open window); near zero means the local air is nearly sealed off (barely any exchange).

3.4 Term 2 — the lake dust source (the heart of the model)

```
E_GSL(t) = beta × A_eff(t) × R(t) × max(U(t) − U_c, 0)^n × D_i(t)
```

This single line encodes everything in Section 2. Building it piece by piece:

PIECE	PLAIN MEANING	TIES BACK TO

`A_eff(t)`	Exposed playa area today, in square kilometers — looked up from the lake's elevation using a real USGS table of how much lakebed is exposed at each possible water level	Section 2.2 — lower lake, more exposed bed
`R(t)`	A number between 0 and 1 that shrinks after rain and recovers as the surface dries back out	Section 2.4 — wet/crusted ground doesn't erode
`max(U(t) − U_c, 0)^n`	Zero below the threshold wind speed `U_c`; above it, grows steeply, controlled by the exponent `n`	Section 2.3 — saltation is a threshold process
`D_i(t)`	A number from 0 to 1 that's largest when wind blows straight from the lake toward the monitor, and zero when it blows the wrong way	Section 2.6 — direction matters
`beta`	An overall "how much dust per unit of everything above" scaling number — this is one of the things the model has to learn from data, since nobody can just measure it directly	—

```
Worked example, using the model's actual fitted numbers (Section 4):   Exposed playa today: A_eff = 600 km^2 Recent rain: none -> R = 1.0 Wind speed: 9 m/s (about 20 mph) Threshold: U_c = 12.3 m/s <- wind is BELOW threshold -> max(9 - 12.3, 0) = 0 -> the whole term is ZERO, no matter how big A_eff is.   Now suppose wind picks up to 15 m/s: max(15 - 12.3, 0)^1.44 = (2.7)^1.44 = ~4.1 E_GSL = 0.16 x 600 x 1.0 x 4.1 x D_i (D_i depends on whether wind is blowing from the lake)
```

This worked example is the whole point of a threshold model: two wind speeds that don't look very different (9 m/s vs. 15 m/s) produce wildly different results — zero vs. a real number — because reality genuinely works that way. A linear "more wind = proportionally more dust" model would get this qualitatively wrong.

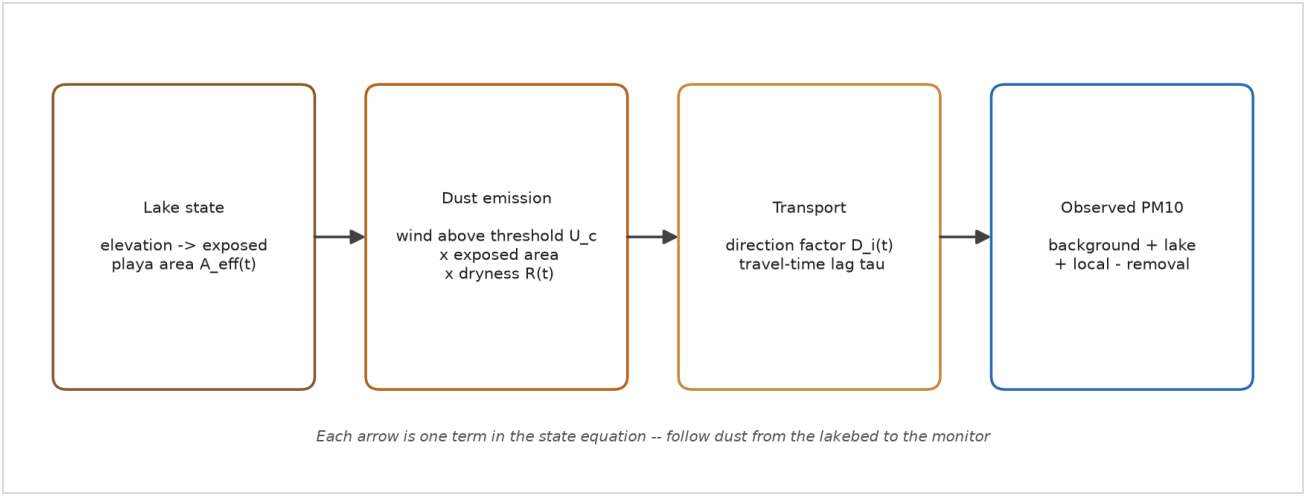
3.5 The travel-time lag

The lake source sits roughly 34 km from the monitor. Dust doesn't teleport — it travels with the wind. The model computes a lag `tau = distance / wind speed` and looks up the emission from `tau` hours ago, not right now. At a brisk 10 m/s wind, that's roughly an hour's delay; in slower wind, several hours.

3.6 Terms 3 and 4 — local sources and removal

```
+ E_local − lambda · P[t]
```

`E_local` is a constant standing in for everyday non-lake sources — traffic, construction, road dust — that add a steady trickle of PM10 regardless of the lake. `lambda` is a removal rate: the fraction of whatever PM10 is currently in the air that settles out, gets deposited, or otherwise disappears each hour, independent of anything else. Together with ventilation (3.3), this is what eventually brings a spike back down after a dust event passes.



The whole pipeline in one picture: lake state determines exposed area, wind and dryness determine whether/how much dust is emitted, wind direction and travel time determine whether/when it reaches the monitor, and it combines with background and local sources (minus removal) to produce what's actually measured.

4. Fitting the model — what "calibration" means

4.1 The dial-turning analogy

The equation in Section 3 has several unknown numbers in it — `beta`, `U_c`, `n`, `lambda`, and a few others. Nobody measured these directly; they have to be **inferred** from data. Picture the equation as a machine with dials on it, one per unknown number. "Fitting" the model means: try a setting of the dials, run the machine on every real hour in the training data, see how far off its predictions are from what was actually measured, then nudge the dials to reduce that error, and repeat thousands of times until the dials stop meaningfully improving. This automatic dial-turning process is called **least squares optimization**.

4.2 A problem we ran into, and how it was fixed

The very first version of this fitting process did something subtly wrong: it turned the wind-threshold dial (`U_c`) almost all the way up, and the "how much dust" dial (`beta`) almost all the way down — effectively deciding "the lake barely matters" and relying on background/local terms for everything.

> **WHY THAT HAPPENED**
>
> Real dust-spike hours are rare — only about 1–2% of all hours. An automatic dial-tuner that's simply told "minimize the average error across every hour" can get a very good average score by nailing the 98% of boring hours and just shrugging at the rare spikes, since a handful of large misses barely move the average. It's the equivalent of a student who only studies the easy questions on a test, because getting a few hard questions wrong doesn't hurt their grade much.

The fix: hours with higher real PM10 were given more weight when scoring the fit, so getting a dust event right (or wrong) actually matters to the dial-tuning process, not just background hours. This is disclosed in the write-up, not hidden, because it's a real methodological choice that shapes every downstream number.

4.3 Uncertainty — the "reshuffle history" idea (bootstrapping)

Once the dials are set, a natural question is: how confident should we be in those exact numbers? If we'd happened to observe slightly different weather over the same years, would the dial-tuner have landed on the same settings, or somewhere quite different?

We can't re-run history, but we can approximate the question: chop the real training data into two-week chunks, randomly draw chunks *with replacement* to build a new, same-length "alternate history" made of real (reshuffled) data, and re-run the entire dial-tuning process on it. Do that 15 times, and look at how much each dial's setting wobbles across those 15 runs. A dial that lands in nearly the same place every time is **well identified** — the data really pins it down. A dial that swings wildly across most of its allowed range every time is **weakly identified** — the data doesn't have a strong opinion about its exact value, even if the overall model still works.

5. How to read the results, and how to tell if they're legit

5.1 The basic accuracy scores: MAE and RMSE

MAE (Mean Absolute Error): take every prediction, subtract the real value, ignore the sign (so being too high and too low count the same), and average. If MAE is 7 μg/m³, the model's typical hourly prediction is off by about 7 micrograms per cubic meter.

RMSE (Root Mean Squared Error): similar, but errors are squared before averaging, then square-rooted at the end. Squaring punishes big misses much more than small ones — so RMSE is the number that tells you how the model handles the large, rare dust spikes specifically, while MAE tells you about typical, everyday performance. When RMSE is a lot higher than MAE for a given model, that model is having occasional big misses on top of otherwise-fine typical performance.

5.2 Why "beat persistence" is the real bar

PM10 an hour from now is usually close to PM10 right now — air quality doesn't usually swing wildly hour to hour. So a trivial "prediction" — just guess whatever the current value is, called the **persistence baseline** — is already a surprisingly strong predictor. Any model that can't beat this lazy guess isn't adding real information; it's just along for the ride. This is why every result in this project is reported next to a persistence score, not in isolation.

5.3 Reading the model ladder table

MODEL	MAE	RMSE	WHAT IT TELLS YOU
Persistence	7.34	11.43	The lazy-guess bar every other model must clear
Physics model	7.39	11.03	Ties persistence on typical hours, **beats** it on large-event hours (lower RMSE) — consistent with a model that specifically understands dust spikes
Hybrid (physics + ML)	6.69	10.59	Best on both counts — the physics structure plus a learned correction layer outperforms either piece alone

> **WHY THIS IS A LEGITIMATELY ENCOURAGING RESULT**
>
> Beating persistence on RMSE specifically — not just MAE — means the physics model is doing better than the naive guess precisely where it's hardest to do better: the rare, large dust spikes. That's exactly the behavior you'd want from a model built around an explicit dust-event mechanism, and it's not guaranteed to come out that way just because the model has more moving parts.

5.4 The single most important legitimacy check: physical sanity checks

Accuracy alone can be misleading — a model can rack up a good score by overfitting to quirks in the training data that have nothing to do with the real mechanism. A much stronger form of evidence is checking whether the model's *internal logic* matches reality on relationships it was never directly told to match.

This project checked three such relationships on data the model never saw while being fit:

- Is the inferred lake contribution actually higher when wind blows from the lake toward the monitor, versus when it doesn't? **Yes.**
- Is it actually lower right after rain, versus during dry stretches? **Yes.**
- Is it actually higher when it's windy, versus calm? **Yes.**

> **WHY THIS MATTERS MORE THAN THE ACCURACY NUMBERS**
>
> Nothing in the fitting process directly forces these three relationships to come out correctly — they emerge from how the fitted numbers interact with real wind, rain, and direction data the model wasn't scored against. A model that got lucky or overfit could easily fail one or more of these. Passing all three is independent evidence the model learned something physically real, not statistical noise.

5.5 Reading "weakly identified" and "at its boundary" warnings

Some fitted numbers (`beta` , `n` , a precipitation-decay setting) either sit right at the edge of the range they were allowed to search, or wobble across most of that range during the bootstrap re-runs (Section 4.3). In plain terms: **the model is confident a lake-dust effect exists and behaves the right way, but not confident about its exact size.** Treat a statement like "peak lake contribution is about 14 µg/m³/hour" as an order-of-magnitude estimate, not a number to defend to two decimal places. This is disclosed directly in the write-up rather than smoothed over, and it's the honest, correct way to report a model fit from one monitor and under five years of data.

5.6 A legitimacy checklist for any future run

Use this list whenever the model is refit on new or extended data, to sanity check the results before trusting them:

1. **Did it beat persistence on RMSE?** If not, something is likely broken or the fit collapsed.
2. **Do all three physical sanity checks still pass** (upwind, dry, windy > their opposites)? If one flips, investigate before trusting anything else.
3. **Which parameters are flagged as "at bounds"?** If suddenly every parameter is pinned (not just the usual weakly-identified few), the fit likely degenerated — check the event-weighting setting first.
4. **Did the held-out test period include real dust events**, or did the time-based split happen to land entirely on quiet months? Check `dust_episodes.csv` against the test date range.
5. **Are the largest "dust" events actually wind-aligned with the lake?** A huge event on a day with wind from the wrong direction is a red flag that it's really wildfire smoke or another source, not a lake event — check the event overlay plots.
6. **Does the counterfactual ordering make sense?** Healthy lake level should always come out lowest; further decline should always come out highest. If that ordering breaks, something upstream is wrong.
7. **Is accuracy suspiciously perfect?** Real hourly air-quality prediction at this level of feature richness should not produce near-zero error — if it does, suspect a data leak (e.g., a future value accidentally included as a "feature").

6. What the model actually found, in plain English

Putting it all together, here is what the current, honestly-caveated model says:

- Great Salt Lake dust **does** show up as a real, physically coherent signal in Salt Lake City's PM10 — it behaves the way basic dust physics says it should (Section 5.4), even though it wasn't told to.
- At the lake's healthiest reasonable level (the state's own 4,198 ft reference), the model finds essentially no lake-dust contribution — which is reassuring, since it means the model isn't inventing a signal out of nothing.
- Between the highest and lowest lake levels actually observed in the 2022–2026 data (a difference of under 7 feet), the model's estimate of lake-driven dust contribution roughly **doubles to triples**.
- A further, hypothetical 5-foot decline below the already-record-low 2022 level would, under this model, push the estimated peak lake contribution roughly 45% higher still.
- The exact size of that effect (the "beta" dial) is not tightly pinned down by the data yet — more years, more monitors, and chemistry/trajectory data (Section 7) would sharpen it considerably. The *direction* and rough *scale* of the finding are much better supported than any specific number.

7. Where to go next

The project's living document (linked from the repository) already has two detailed follow-on plans:

- **A tiered roadmap** toward a best-in-class model — what's achievable solo (real HRRR mixing-height data, multi-site monitors, bay-specific lake sources, a Bayesian refit with literature-informed priors), versus what genuinely needs institutional resources (chemistry labs, HPC dispersion modeling, field measurements).
- **A public-data maximization plan** — a concrete, source-by-source list of free data (PM2.5 for dust-vs-smoke separation, more air-quality monitors, satellite dust/smoke detection, free back-trajectory modeling, satellite surface classification of the playa) with a suggested pull order, all following the same code pattern already established in the project.

A good way to build hands-on intuition before diving into either plan: open `src/gsl_model/config.py` first (it's short and mostly constants), then `physics_model.py` (the equation from Section 3, in code), then run `scripts/run_pipeline.py` end to end once by hand. Then try one small experiment: change `event_weight_power` in the pipeline script (Section 4.2) to 0, rerun, and watch `U_c` and `beta` collapse toward the degenerate values described in that section. Seeing that failure mode happen on purpose is one of the fastest ways to really understand why the weighting choice was necessary.

This document was generated alongside the project's code and living document from the same September 2026 pipeline run; all figures and numbers here match `results/tables/` and `docs/MODEL_EXPLAINER.md` in the repository.